SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1021	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Bhuvaneshwaran J	**Reg. No.:**	192511402
Date:		**Duration:**	60 minutes

Code of Conduct

I Bhuvaneshwaran J(192511402) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Bhuvaneshwaran J

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
   - Describe the existing system or current process.
   - Identify its strengths and limitations.
   - Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.
   - Describe the major functional modules of the proposed system.
   - Identify the functional and non-functional requirements.
   - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**
   - Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).

- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/frame work and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks

				organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

Industry Context: International banking transfers via legacy networks like SWIFT remain slow, expensive, and non-transparent, taking days to settle across different time zones, clearinghouses, and currency pairings. These delays trap billions of dollars in dormant capital within intermediary banks to cover settlement risks. A real-time, cross-border transactional architecture must bypass regional silos while maintaining precise compliance, liquidity management, and foreign exchange execution.

Problem Statement: Design and develop a distributed, high-availability Cross-Border Settlement & Liquidity Routing Protocol that executes multi-currency financial transactions instantly. The system must monitor real-time treasury balances, foreign exchange volatility, regional compliance blacklists, and transaction fees across global liquidity hubs. It must dynamically determine the most capital-efficient path to route, convert, and settle a transaction across international borders within seconds.

Objectives:

Coordinate
multi-currency transaction state transitions with zero-error atomic consistency.

Optimize
transaction routing paths across global banking nodes using dynamic graph weights.

Integrate
automated Sanction Screening and AML checks within the transaction lifecycle.

Maintain
highly accurate, synchronized ledgers across geographically distributed nodes.

Formulate
fallback strategies for instant transaction reversal during node connectivity drops.

Expected Outcomes:

Reduction
of cross-border settlement overhead times from days to sub-second windows.

Minimized
foreign exchange exposure risks for multinational enterprise transactions.

Elimination
of manual reconciliation efforts between intermediary banking systems.

Lower
operational fees passed down to international remittance senders.

1. UNDERSTAND THE PROBLEM STATEMENT

1.1 Problem in My Own Words

The assigned problem concerns international money transfers that depend on legacy networks and multiple intermediary institutions. A transfer may need to move through several banks before reaching the beneficiary bank. Each intermediary can introduce processing queues, operating-hour constraints, fees, foreign-exchange spreads, compliance checks, and reconciliation work. Because banks cannot always know the status and liquidity position of every other participant in real time, they may hold additional funds as a precaution.

The proposed system aims to coordinate the transfer as a single managed transaction while selecting an efficient path through available banking or payment nodes. It must compare route cost, foreign-exchange rate, available liquidity, estimated latency, node health, and compliance eligibility. It must also maintain a reliable record of each state change and recover safely when a network connection fails.

1.2 Core Problem

The core engineering problem is not simply moving a message faster. It is coordinating a multi-party financial transaction across independent institutions without creating duplicate payments, losing ledger consistency, bypassing compliance controls, or committing funds that are not available. The system must distinguish between a request being accepted, liquidity being reserved, a payment being submitted, external settlement being confirmed, and a transaction being reversed or placed under review.

1.3 Objectives

- Coordinate multi-currency transaction state transitions with atomic, auditable consistency.
- Optimize routes across global banking nodes using dynamic graph weights.
- Integrate sanctions screening and anti-money-laundering (AML) checks into the transaction lifecycle.
- Maintain synchronized, highly accurate ledgers across geographically distributed nodes.
- Provide safe fallback and compensating actions during node or network failure.
- Reduce avoidable settlement delay, FX exposure, manual reconciliation and transfer overhead.

1.4 Scope of the Proposed System

The system scope includes payment initiation through authorized APIs, identity and message validation, compliance screening, treasury and liquidity checks, foreign-exchange quote selection, graph-based route calculation, transaction-state coordination, payment-rail adapters, ledger events, reconciliation, alerts, and operational reporting. It supports a configurable set of currencies, participating institutions, settlement corridors, transaction limits, and jurisdiction-specific policies.

In Scope

- Multi-currency transfers between onboarded institutions and supported corridors.
- Live or frequently refreshed liquidity and FX inputs, with timestamps and data-quality checks.
- Route selection using cost, FX spread, available liquidity, risk, latency and reliability.
- Sanctions/AML screening with reject, hold, or manual-review outcomes.
- Idempotent transaction APIs, durable event records, audit trails and reconciliation.
- High-availability deployment, monitoring, incident response and disaster recovery.

Out of Scope / Boundary Conditions

- Replacing every national payment system or eliminating all external settlement dependencies.
- Guaranteeing sub-second final settlement where an external rail, legal process or institution is slower.
- Autonomous approval of every AML alert; complex cases may require compliance officer review.
- Operating without licensing, institutional agreements, access controls or jurisdictional approvals.
- Treating distributed ledger replication as a substitute for legal settlement finality.

1.5 Assumptions and Constraints

The design assumes participating banks expose authenticated interfaces or approved message adapters, provide sufficiently current treasury data, and agree on transaction identifiers and status semantics. Each corridor requires a documented legal and operational model. Some currencies may have capital controls, limited operating windows, or restricted convertibility. Sanctions lists, risk rules, retention requirements and data-localization obligations vary by jurisdiction and must be configurable. Therefore, the protocol can optimize only among paths that are currently eligible, permitted, and supported.

2. IDENTIFY KEY STAKEHOLDERS AND CHALLENGES

Cross-border settlement involves commercial, technical, regulatory, and operational participants. A successful design must make each party's authority explicit, avoid exposing confidential information unnecessarily, and provide a traceable record of who or what approved each state transition.

Stakeholder	Role and responsibilities	Main challenges
Sending bank / payment originator	Authenticates customer, initiates payment, supplies purpose and beneficiary details	Status uncertainty, fees, rejected payments, customer support
Receiving / beneficiary bank	Validates beneficiary account and confirms credit according to local rules	Incoming-message quality, local cut-off times, reconciliation
Correspondent / liquidity provider banks	Provide accounts, currency liquidity, FX and settlement connectivity	Prefunding cost, credit exposure, nostro/vostro reconciliation
Treasury and FX desk	Publishes balances, limits, executable quotes and risk thresholds	Volatile FX markets, stale balances, liquidity concentration
Compliance / AML team	Maintains screening rules, reviews alerts, documents decisions	False positives, evolving lists, explainability and auditability
Operations and reconciliation team	Monitors exceptions, matches records, resolves uncertain transactions	Manual investigations, duplicate records, inconsistent status
Platform operator / SRE	Maintains uptime, observability, incident response and recovery	Regional outages, capacity, recovery objectives
Regulators / auditors	Set and examine legal, prudential, reporting and recordkeeping obligations	Evidence quality, jurisdictional differences, control effectiveness
Enterprise / remittance customer	Submits payment instructions and receives status/fee information	Slow delivery, opaque charges, FX uncertainty
Security and data-protection teams	Manage identities, keys, access, privacy and incident controls	Fraud, credential theft, sensitive-data exposure

2.1 Stakeholder Coordination

The platform should use role-based access and explicit ownership. Treasury owns liquidity limits and quote policies; compliance owns screening rules and disposition of alerts; operations owns exception handling; engineering owns service reliability; and participating banks remain responsible for their regulated activities and settlement obligations. A transaction must not be marked as finally settled merely because the platform successfully sent a message.

2.2 Major Challenges and Issues

Liquidity visibility
A routing engine needs a reliable view of spendable liquidity, not merely a balance snapshot. Balances may change between quote generation and payment execution. Each balance must therefore include source, timestamp, currency, institution, account or pool, reserved amount, available amount, and confidence or freshness status. A stale or uncertain balance should be excluded or penalized.

Foreign-exchange volatility
Rates can move between calculation and execution. The platform should request executable quotes, record the quote identifier and expiry, and define whether the transaction is repriced, rejected, or rerouted when the quote expires. FX cost must include spread and any applicable conversion fee, not just the displayed mid-market rate.

Compliance and privacy
Sanctions screening and AML monitoring can require sensitive customer and transaction data. The architecture should minimize data sharing, encrypt traffic and stored records, restrict access, log decisions, and preserve manual-review controls. A routing optimization must never override a compliance block.

Distributed consistency and failure
Nodes can fail after receiving a request but before returning a response. The sender may not know whether the payment was accepted externally. Blindly retrying can create duplicate payments. The design therefore uses unique transaction identifiers, idempotency keys, durable state transitions, status queries, reconciliation, and compensating actions where reversal is legally and technically possible.

Governance across institutions
Independent banks may use different data formats, operating calendars, service-level commitments, and dispute procedures. Common schemas, versioned APIs, settlement-status definitions, escalation paths, and participation agreements are essential. The platform should support corridor-specific rules rather than assume that one workflow is valid worldwide.

3. ANALYZE THE CURRENT APPROACH

3.1 Existing Process

A conventional cross-border transfer may begin when a customer instructs a sending bank to pay a beneficiary in another country. The sending bank validates the instruction and applies its controls. The payment message may travel through a financial messaging network, then through one or more correspondent banks that hold accounts in relevant currencies. Intermediaries may debit and credit accounts, convert currencies, apply fees, and forward instructions to the next institution. The receiving bank applies its own checks and credits the beneficiary when the applicable conditions are met.

The messaging network communicates instructions and status information, but messaging itself is not the same as settlement. Actual movement of funds depends on account relationships, settlement arrangements, local payment systems, liquidity, cut-off times, and the participating institutions' processes. Operations teams may later reconcile statements, payment messages and internal ledgers.

3.2 Existing Approach Workflow

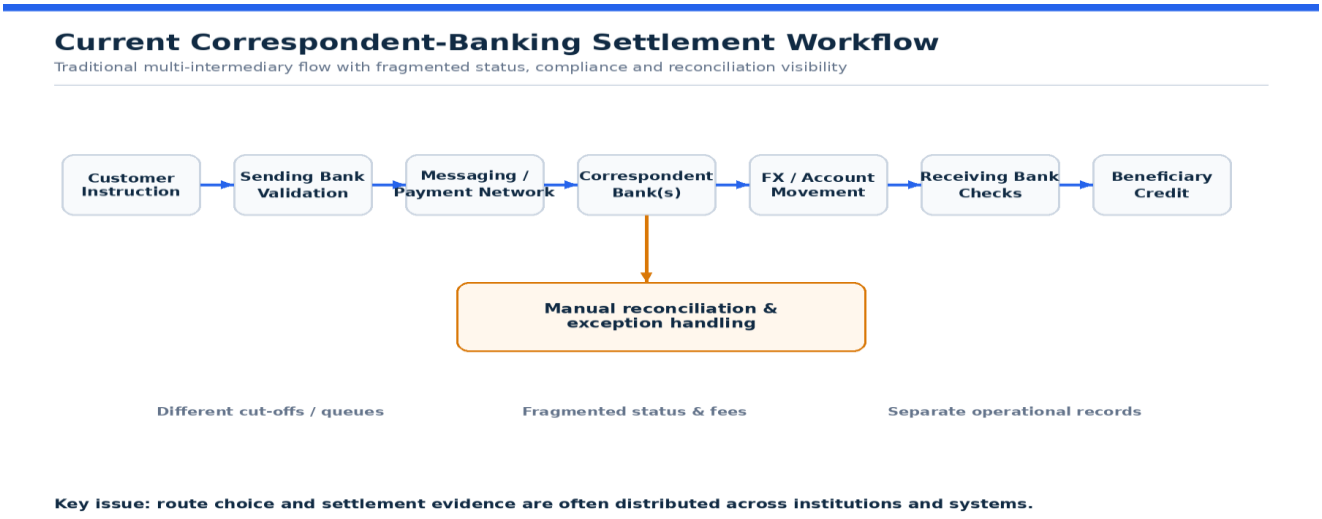
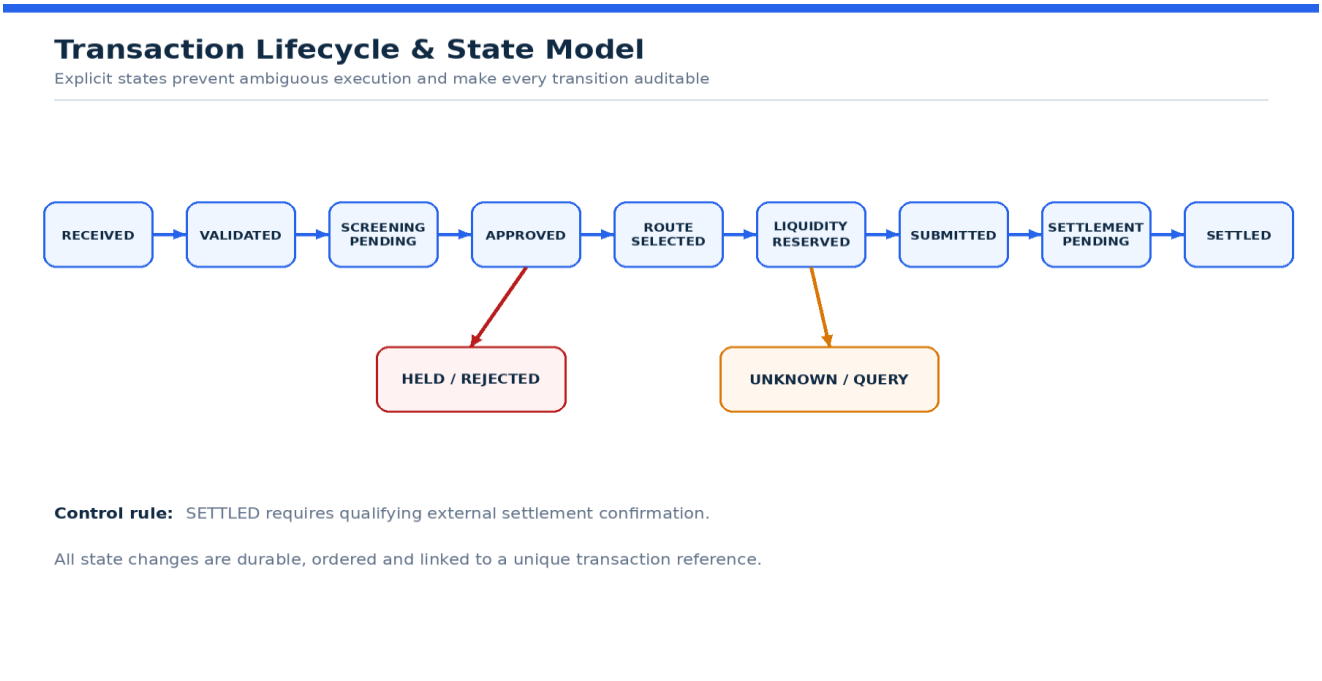
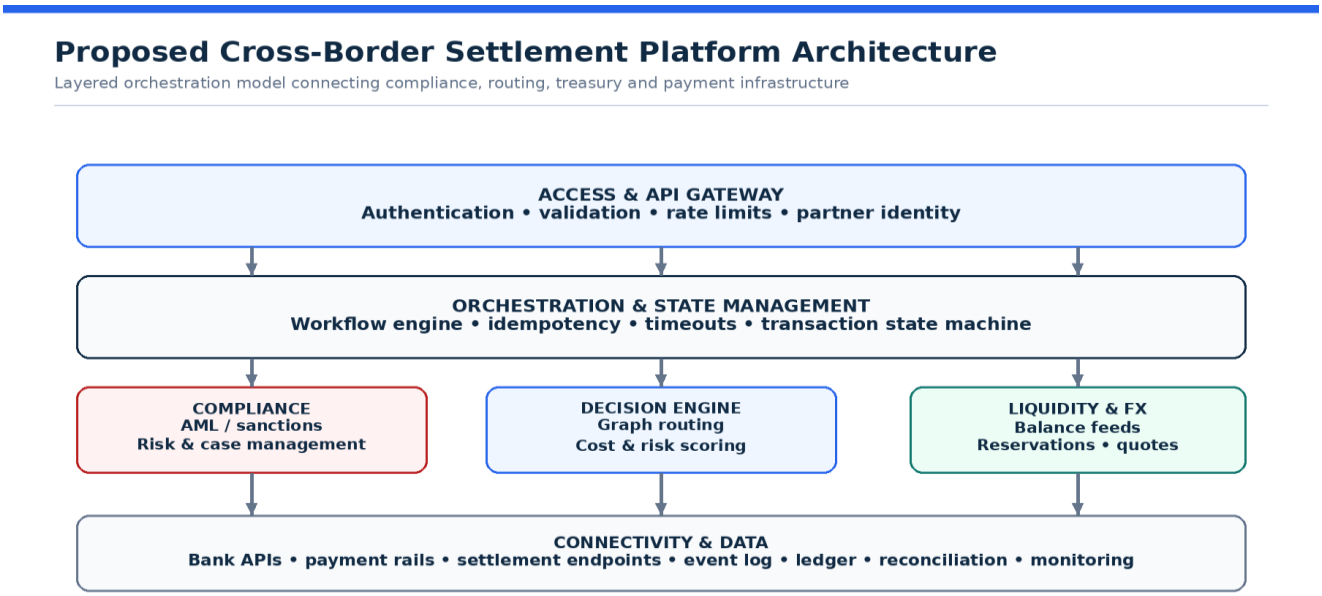


Figure 3.1. Current Correspondent-Banking Settlement Workflow





Figures 4.1 and 4.2 present the transaction lifecycle/state model and the proposed platform architecture of the target design.

3.3 Strengths of the Current Approach

- Established institutional relationships, messaging practices and operational controls.
- Broad international reach through participating banks and payment arrangements.
- Mature compliance processes and documented responsibilities within individual institutions.
- Existing accounting, audit and reconciliation systems that support regulated operations.

3.4 Limitations and Gaps in the Current Approach

Issue	Why it occurs	Effect on users / banks
Long or variable processing time	Multiple institutions, time zones, cut-off times and manual queues	Uncertain delivery time and working-capital delay
Opaque end-to-end status	Different institutions expose different status and reference data	Customer support effort and difficult investigations
Liquidity trapped in accounts	Banks prefund accounts to meet uncertain payment needs	Higher opportunity cost and treasury complexity
Multiple FX conversions	Route may involve intermediary currency conversion	Cumulative spreads and increased FX exposure
Manual reconciliation	Different statements, formats and timing	Operations cost and delayed exception resolution
Limited dynamic routing	Routes may follow fixed correspondent relationships	Suboptimal fees, latency or liquidity usage
Failure ambiguity	A timeout does not prove that a payment failed	Duplicate-payment risk or delayed recovery
Fragmented compliance evidence	Screening and decisions may be distributed across institutions	Repeated checks, inconsistent records and audit burden

3.5 Engineering Gap

The gap is the lack of a unified, policy-controlled orchestration layer that can evaluate multiple eligible routes with current liquidity and FX information while preserving compliance gates, explicit transaction states, and recoverable execution. The proposed system does not eliminate correspondent banks or local payment rails by definition. Instead, it seeks to improve coordination among connected institutions and select a more efficient supported path.

3.6 Comparative Analysis: Current Approach vs Proposed Protocol

The comparison below evaluates the existing correspondent-banking approach against the proposed Cross-Border Settlement & Liquidity Routing Protocol using the report's stated problem areas, requirements and engineering objectives. The proposed system is an orchestration and decision layer; it does not assume that correspondent banks or external payment rails disappear.

Comparison focus: the current process prioritizes established correspondent relationships and institution-specific controls, while the proposed protocol adds a policy-controlled orchestration layer for dynamic route selection, liquidity/FX awareness, explicit transaction states, reconciliation and safe recovery.

Evaluation area	Current approach	Proposed protocol	Expected improvement
Routing	Often follows established correspondent relationships; limited dynamic path comparison.	Graph-based evaluation ranks eligible paths using fees, FX cost, latency, liquidity and risk.	More explainable and adaptive route selection.
Liquidity	Prefunding and balance visibility can be fragmented or stale.	Freshness-aware liquidity feeds, reservations and treasury limits are incorporated into routing.	Better use of available liquidity and lower avoidable buffers.
FX	Intermediary conversions may create cumulative spreads and exposure.	Executable quotes, expiry checks and FX cost are considered during route evaluation.	Greater FX-cost transparency and controlled exposure.
Compliance	Controls exist within individual institutions; evidence can be fragmented.	Screening is an explicit lifecycle gate with hold/reject/manual-review outcomes and audit evidence.	More consistent control and traceability.
Transaction state	Status semantics vary between	Durable state machine with explicit	Lower duplicate-payment and

	institutions; timeout can create ambiguity.	UNKNOWN, RETRYABLE and compensation states.	status-reconciliation risk.
Reconciliation	Multiple statements, formats and timings create manual work.	Canonical transaction IDs, durable events and reconciliation queues.	Faster exception detection and lower manual effort.
Failure recovery	Recovery depends heavily on participant-specific processes.	Idempotency, status queries, circuit breakers, bounded retries and compensating actions.	Safer recovery from network and partner failures.
Auditability	Evidence may be distributed across banks and operational systems.	Correlated, tamper-evident events record state changes, decisions and external references.	Improved audit reconstruction and accountability.
Customer visibility	End-to-end status and fee/FX information may be fragmented.	Unified status history and explainable route/fee information for authorized users.	Clearer customer communication.
Integration & scalability	Mature network reach but heterogeneous interfaces and processes.	Versioned adapters, common schemas and horizontally scalable orchestration services.	Easier controlled expansion across supported corridors.

3.7 Why a New Software Solution Is Needed

A centralized decision service alone could become a single point of failure and may not suit institutions requiring regional control. Conversely, unrestricted peer-to-peer execution would make governance, auditability and consistent state management difficult. A practical design combines logically coordinated transaction orchestration with replicated durable records, regionally deployable services, institution-specific adapters, and controlled access to liquidity and compliance data.

4. PROPOSE A SUITABLE SOFTWARE SOLUTION

4.1 Solution Overview

The proposed Cross-Border Settlement & Liquidity Routing Protocol is a secure, distributed orchestration platform. A transaction is accepted only after schema validation, authentication, limits checks and compliance evaluation. The routing engine constructs candidate paths across participating nodes, removes ineligible paths, and scores remaining paths using total fees, FX spread, liquidity availability, expected latency, node health, and risk limits. The selected route is executed with a quote and liquidity reservation where supported. Every state change is persisted and auditable.

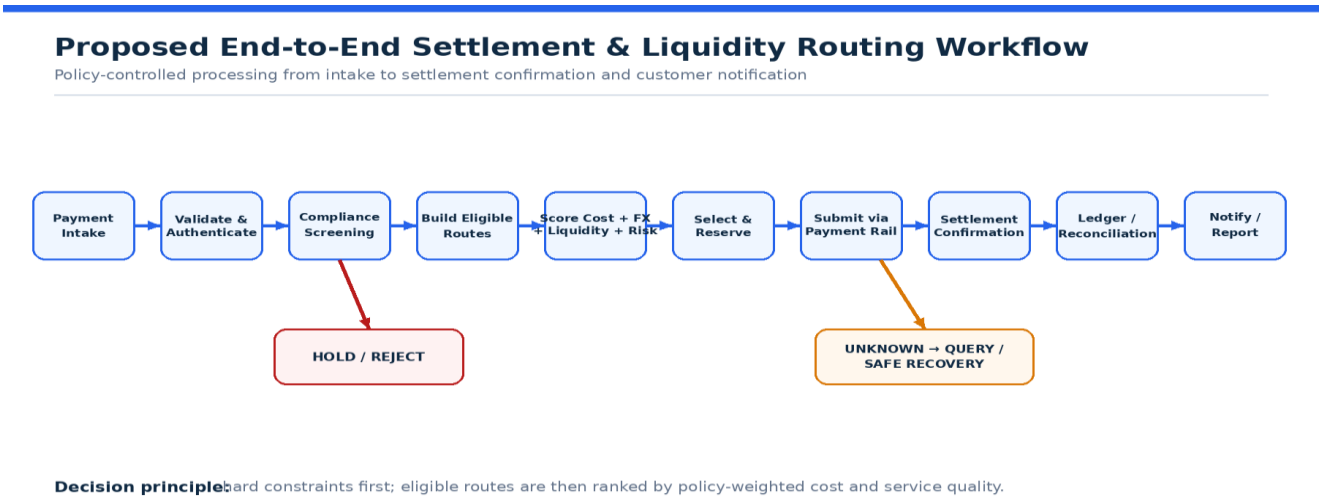


Figure 4.3. Proposed End-to-End Cross-Border Settlement & Liquidity Routing Workflow

4.2 Architectural Layers

- Access layer: authenticated API gateway, request validation, rate limiting and partner identity.
- Orchestration layer: transaction state machine, idempotency, workflow coordination and timeout handling.
- Compliance layer: sanctions screening, AML rules, risk scoring, case management and audit evidence.
- Decision layer: graph-based route search, cost model, route constraints and explainable decision output.
- Liquidity and FX layer: balance feeds, reservations, executable quotes, quote expiry and treasury limits.
- Connectivity layer: adapters to bank APIs, payment rails, settlement systems and institution-specific formats.
- Data and control layer: transaction ledger, event log, reconciliation, monitoring, reporting and disaster recovery.

4.3 Major Functional Modules

Module	Responsibilities	Key outputs
Payment intake	Validate sender, beneficiary, amount, currency, purpose, idempotency key and authorization	Accepted or rejected request
Compliance screening	Screen parties and transaction data against relevant lists and AML policies	Clear, hold, reject or review decision
Treasury monitor	Ingest balances, reservations, limits and data freshness from participating nodes	Spendable liquidity view
FX quote service	Obtain quotes, record spread, fees, provider, expiry and quote conditions	Executable conversion option
Route optimizer	Build candidate paths, enforce constraints, score paths and explain selected route	Ranked eligible route
Transaction coordinator	Manage state machine, timeouts, retries, reservation and compensation	Durable transaction state
Ledger and event store	Persist immutable or tamper-evident events and maintain accounting references	Auditable transaction history
Payment adapters	Translate common commands to bank/rail-specific APIs and message formats	External payment instruction and response
Reconciliation service	Compare internal state with partner acknowledgements and statements	Matched, unmatched or disputed items
Operations console	Display status, alerts, exceptions, manual review and incident context	Controlled operational action
Security and audit	Manage identities, keys, access policies, logs and evidence retention	Access decisions and audit trail

4.4 Transaction State Model

A suggested state model is RECEIVED → VALIDATED → SCREENING_PENDING → APPROVED or HELD/REJECTED → ROUTE_SELECTED → LIQUIDITY_RESERVED → SUBMITTED → SETTLEMENT_PENDING → SETTLED. Failure branches include RETRYABLE, UNKNOWN, COMPENSATION_PENDING, REVERSED, and MANUAL_REVIEW. Only authorized transitions are allowed. Each event records transaction ID, previous and new state, timestamp, actor/service, reason code, correlation ID and related external reference.

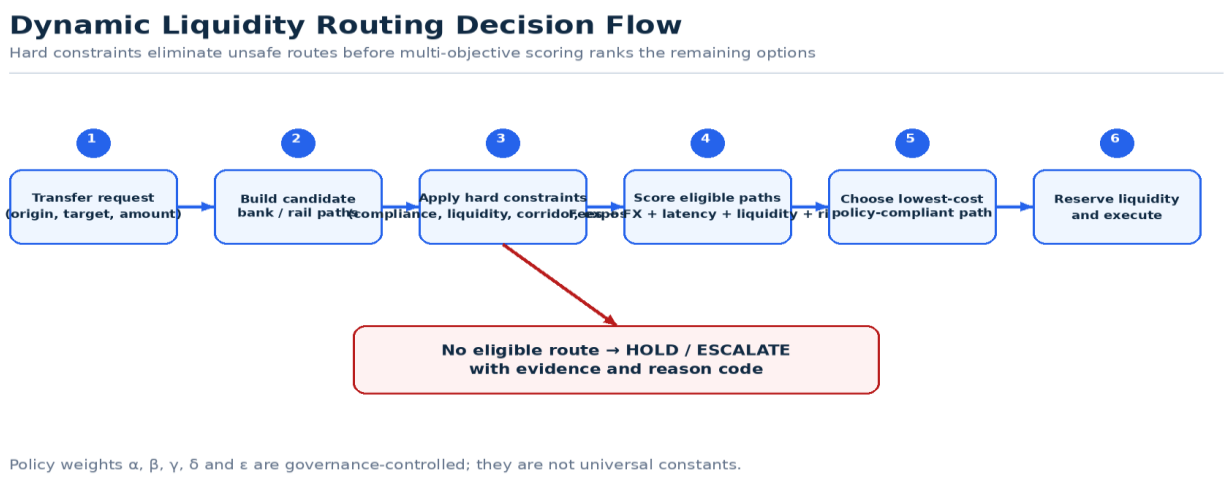


Figure 4.4. Dynamic Liquidity Routing Decision Flow

The state machine should reject impossible transitions—for example, a transaction cannot be marked SETTLED before a qualifying settlement confirmation. State changes must be durable before a success response is returned where that response implies the platform has accepted responsibility for processing.

4.5 Dynamic Graph-Based Liquidity Routing

Represent the participating financial network as a directed graph G = (V, E). Each vertex represents an institution, liquidity hub, currency pool or supported settlement endpoint. Each directed edge represents a feasible transfer, conversion, or settlement operation. Edge attributes may include fee, FX spread, estimated latency, liquidity capacity, operational availability, credit exposure, corridor restrictions, and compliance eligibility.

A simplified route cost can be expressed as:

$$Cost(path) = \alpha \cdot Fees + \beta \cdot FXCost + \gamma \cdot Latency + \delta \cdot LiquidityPenalty + \varepsilon \cdot RiskPenalty$$

The coefficients α, β, γ, δ and ε are policy weights chosen by treasury and risk governance. They are not universal constants. Hard constraints—such as sanctions blocks, insufficient liquidity, prohibited corridors, maximum exposure, and expired quotes—must eliminate a path rather than merely increase its score. Soft preferences can then rank eligible paths.

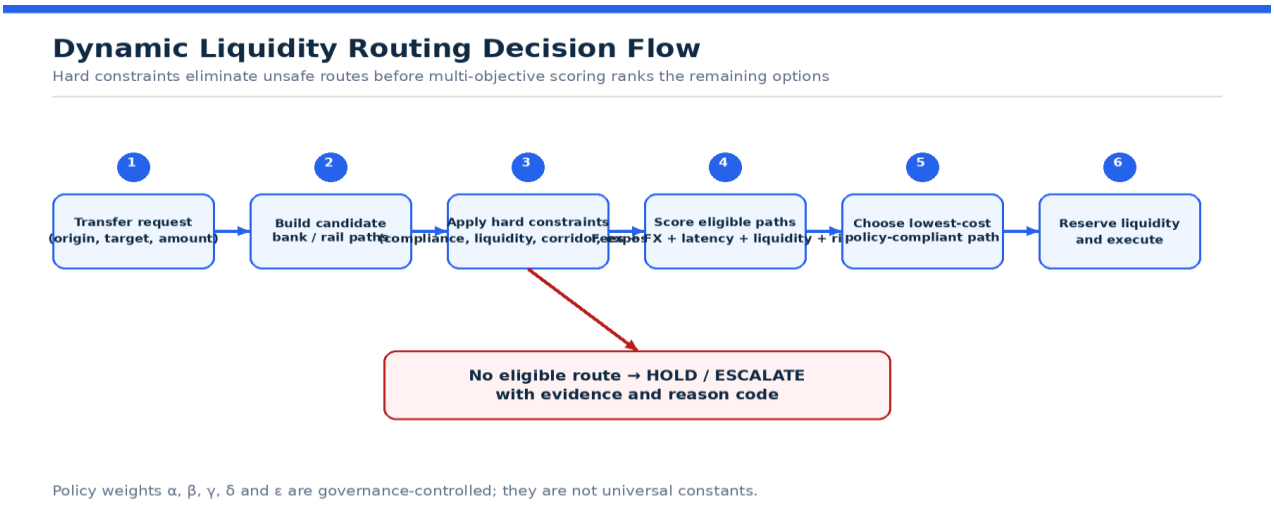


Figure 4.4. Illustrative Dynamic Route-Selection Decision Flow

4.6 Example Route Evaluation

Candidate	Fees (illustrative)	FX cost (illustrative)	Latency estimate	Liquidity / risk	Decision
Route A: direct partner	$8	$12 equivalent	2 seconds	Sufficient / low	Eligible
Route B: two intermediaries	$5	$18 equivalent	8 seconds	Sufficient / medium	Eligible, higher total cost
Route C: liquidity hub	$4	$7 equivalent	3 seconds	Balance stale	Exclude until refreshed

These figures are hypothetical examples to explain the decision method, not market quotes or measured project results. The route optimizer should compare total expected cost and service quality using current data. It must not choose Route C simply because its displayed fee is lowest when its balance information is stale.

4.7 Functional and Non-Functional Requirements

Functional Requirements

ID	Requirement	Acceptance evidence
FR-01	Accept authenticated transfer instructions in supported currencies and corridors	Valid request creates a unique transaction record
FR-02	Validate sender, beneficiary, amount, purpose, limits and idempotency key	Invalid or duplicate request is safely rejected or returns existing result
FR-03	Run required sanctions and AML controls before execution	Flagged transactions are held/rejected according to policy
FR-04	Retrieve current treasury balances and enforce liquidity limits	Ineligible or stale liquidity is not treated as available
FR-05	Obtain and validate executable FX quotes	Expired quote cannot be used without re-quotation
FR-06	Rank eligible routes using configured graph weights	Decision includes route, score, input timestamp and explanation
FR-07	Record every state transition and external reference	Auditor can reconstruct lifecycle from durable events
FR-08	Retry safely and reconcile uncertain outcomes	Retries do not create duplicate transfers
FR-09	Support operator review and exception resolution	Manual actions require authorization and audit reason
FR-10	Provide status notifications and operational reports	Authorized user can retrieve current status and history

Non-Functional Requirements

Quality attribute	Proposed target / control
Availability	Initial service target: 99.95% monthly for core orchestration, subject to design and budget validation
Performance	Internal route decision target: p95 under 200 ms with warmed data; external settlement excluded from this target
Consistency	Durable, ordered state transitions per transaction; no duplicate logical execution
Security	Strong service identity, least privilege, encryption in transit and at rest, key rotation
Scalability	Horizontal scaling of stateless APIs and workers; partitioned event processing
Recoverability	Defined RTO/RPO, tested backups and regional recovery procedures
Auditability	Correlated, tamper-evident event trail and configurable retention
Maintainability	Versioned APIs, modular adapters, automated tests and deployment pipelines

These targets are proposed engineering objectives for validation during design and load testing, not measured results or a guarantee of production performance.

4.8 Software Engineering Principles

Modularity and separation of concerns

Payment intake, compliance, route calculation, treasury, execution, reconciliation and reporting are separate modules with documented contracts. A change to one bank adapter should not require

rewriting the route optimizer. Shared transaction identifiers and event schemas allow modules to collaborate without sharing all internal implementation details.

Scalability

Stateless API services can scale horizontally. Work queues and event streams can distribute independent transactions, while per-transaction ordering and idempotency protect state correctness. Data partitioning should preserve the ability to reconstruct an individual transaction and support cross-region reporting. Capacity must be tested for peak volumes, partner bursts and retry storms.

Reliability and availability

Use health checks, timeouts, circuit breakers, bounded retries, queue back-pressure, redundant service instances and regional failover. A circuit breaker prevents repeated calls to an unhealthy node. Retries should use exponential backoff and jitter and must be idempotent. Availability design should not weaken compliance or accounting controls.

Security and privacy

Use mutual TLS or equivalent authenticated channels for partner traffic, short-lived credentials, secrets management, strong administrator authentication, least-privilege roles, encryption at rest, key rotation, and secure software supply-chain practices. Sensitive customer data should be minimized and masked in operational logs. Security events and privileged changes should be auditable.

Maintainability and usability

Versioned APIs, clear error codes, configuration validation, automated regression tests and documented runbooks make the platform maintainable. An operations console should show the current state, last successful event, missing acknowledgement, selected route, reason for a hold, and safe next actions. Operators must not be able to override sanctions blocks or accounting controls without a legally approved process.

Atomic consistency across distributed systems

A distributed transaction cannot assume that a database transaction spans every external bank and payment rail. The design should use durable local transactions, an event log, idempotent commands, explicit acknowledgements, and a saga-style workflow with compensating actions. Where supported, liquidity can be reserved before submission. Reversal is not always equivalent to rolling back a database write: once funds settle, recovery may require a separate return payment or formal dispute process.

4.9 Security, Compliance and Data Governance

Compliance controls are mandatory gates in the transaction lifecycle. The system should screen relevant parties and transaction attributes using current jurisdiction-specific lists and institution-approved rules. Potential matches should be assigned a reason code and routed to a hold or review queue. Automated screening results, list versions, timestamps, rule versions, reviewer actions and final dispositions should be retained according to policy.

Key controls

- Authentication and authorization for customers, partner institutions, services and administrators.
- Encryption for data in transit and at rest, with managed keys and rotation procedures.
- Data minimization: only disclose fields needed by a specific participant or compliance function.
- Sanctions and AML rules versioned by jurisdiction, with controlled approval and rollback.
- Transaction limits, velocity controls, anomaly detection and escalation for suspicious patterns.
- Immutable or tamper-evident audit records with synchronized time sources and correlation IDs.
- Segregation of duties for rule changes, manual reviews, treasury limits and production deployment.
- Security monitoring, vulnerability management, penetration testing and incident notification procedures.

4.10 Ledger and Reconciliation Design

Each transaction should have a canonical internal identifier and preserve external references for every institution and rail. The ledger/event store records the platform's view of events, while partner confirmations and statements provide evidence of external status. Reconciliation compares expected and actual movements, identifies unmatched items, and assigns them to an exception queue. A distributed ledger may be considered where multiple institutions need shared, tamper-evident records, but it is not automatically required; a replicated database and append-only event log may be simpler and more efficient.

Ledger accuracy depends on clear semantics: initiated, accepted, reserved, submitted, settled, returned, reversed and disputed are different states. Reports should distinguish platform processing time from external settlement time.

5. JUSTIFY THE CHOSEN METHODOLOGY / FRAMEWORK

5.1 Selected Methodology: Agile Scrum with DevSecOps

Agile Scrum is suitable because the system has several complex, changing domains: institution connectivity, compliance policy, liquidity and FX integration, reliability, security, and operational workflows. These components cannot be fully validated by writing a long specification and postponing feedback until the end. Scrum enables incremental delivery, review with stakeholders, and reprioritization as technical and regulatory constraints become clearer.

DevSecOps complements Scrum by integrating automated testing, dependency and container scanning, infrastructure-as-code review, controlled deployment, observability and security checks into the delivery pipeline. Because financial software is high impact, Agile must not mean uncontrolled change. Every increment needs documented acceptance criteria, security review, test evidence, traceability and approval before production release.

5.2 Scrum Roles

Role	Responsibilities in this project
Product Owner	Prioritizes outcomes, maintains backlog, defines acceptance criteria with treasury, compliance, operations and bank partners
Scrum Master	Facilitates events, removes blockers, supports delivery transparency and continuous improvement
Developers / engineering team	Builds services, adapters, tests, infrastructure, documentation and operational tooling
Domain stakeholders	Treasury, compliance, operations, security, partner banks and legal experts validate requirements and risks

5.3 Proposed Sprint Plan

Sprint	Primary deliverables
Sprint 0 / Discovery	Corridor selection, stakeholder interviews, risk assessment, architecture decisions and measurable acceptance criteria
Sprint 1	API gateway, identity, transaction schema and idempotency foundation
Sprint 2	Transaction state machine, durable event log and audit trail
Sprint 3	Treasury data ingestion, freshness controls and liquidity limits
Sprint 4	FX quote adapter and initial graph-based route scoring
Sprint 5	Sanctions/AML integration, hold/reject workflows and review console
Sprint 6	Payment-rail adapter, reconciliation and safe retry behavior
Sprint 7	Resilience, security testing, load testing and operational runbooks
Pilot / hardening	Controlled corridor pilot, defect remediation, audit review and go-live decision

5.4 Why Agile Scrum Is Appropriate

- Incremental risk reduction: state management and idempotency can be tested before multiple live corridors are added.
- Frequent stakeholder feedback: treasury can validate cost weights; compliance can validate screening disposition; operations can validate exception handling.

- Changing requirements: new sanctions rules, partner API changes, currency constraints and local regulations can be added through controlled backlog refinement.
- Early integration: payment-rail and FX adapters can be tested with simulators and sandbox endpoints before production connectivity.
- Measurable quality: each sprint can include automated tests, threat-model updates, performance measurements and acceptance demonstrations.
- Maintainable delivery: small changes, versioned interfaces and automated deployment reduce the risk of large, difficult releases.

5.5 Definition of Done and Governance

A feature is complete only when its acceptance criteria pass, unit and integration tests succeed, security checks are reviewed, observability is added, failure paths are tested, documentation is updated, and an accountable stakeholder accepts the behavior. Changes to compliance rules, route weights, liquidity limits and transaction-state transitions require approval and traceability. Emergency changes need a documented incident reason and post-implementation review.

5.6 Maintenance and Release Strategy

Use separate development, test, staging and production environments. Partner sandboxes and simulated payment rails should be used for routine testing. Deploy gradually using canary or blue/green release techniques where possible, monitor latency and error rates, and provide a rollback plan. Database and event-schema changes must be backward compatible during mixed-version deployment. Regular disaster-recovery exercises and incident simulations are part of maintenance, not optional final activities.

5.7 Alternative Methodologies

Method	Assessment for this project
Waterfall	Useful for fixed, contractually specified deliverables, but feedback and integration risks may surface too late.
Spiral	Strong risk focus and useful for early prototypes; can be heavier to manage for every incremental feature.
Pure DevOps	Excellent delivery and operations practices but does not alone provide product-planning and stakeholder-review structure.
Agile Scrum + DevSecOps	Selected combination: iterative planning, stakeholder review, continuous quality and controlled operational release.

6. EXPECTED OUTCOMES AND BENEFITS

6.1 Expected Benefits

The intended benefit is a more coordinated and transparent transfer lifecycle. The platform can reduce internal decision delays by evaluating route options automatically, using fresh liquidity and FX inputs, and maintaining one traceable transaction history. The largest gains are expected in eligible corridors where participating institutions provide timely data and compatible execution interfaces.

- Faster orchestration: automated validation, screening, route selection and status updates reduce avoidable queueing.
- Improved cost visibility: route decisions account for fees and FX cost, making trade-offs explainable.
- Better liquidity use: reservations and live balance feeds can reduce unnecessary buffers, subject to treasury policy.
- Lower manual reconciliation effort: common identifiers and structured events make unmatched transactions easier to detect.
- Improved operational resilience: explicit states, safe retries and recovery workflows reduce ambiguity after outages.
- Stronger compliance evidence: rule versions, screening decisions and reviewer actions are recorded for audit.
- Better customer experience: more consistent status information and clearer fee/FX disclosures.

6.2 Expected Outcome Measures

Measure	Proposed evaluation method	Caution
Internal routing latency	Measure p50/p95/p99 from validated request to route decision	Does not equal final external settlement time
End-to-end settlement time	Compare timestamps from initiation to confirmed settlement by corridor	Depends on external rails and participant operating windows
Total transfer cost	Add intermediary fees, FX spread and platform-related cost for comparable transfers	Use like-for-like currencies, amounts and service levels
Reconciliation effort	Track manual cases and staff time per 1,000 transactions	Baseline is needed before claiming improvement
Liquidity utilization	Measure average prefunding and idle balances under treasury controls	Changes may reflect volume or risk policy, not only software
Failure recovery	Inject failures and measure safe resolution time and duplicate count	Target duplicate logical execution count is zero
Compliance quality	Review screening coverage, false positives, false negatives and case handling	Must be assessed by qualified compliance staff

6.3 Performance Targets

Initial engineering targets may include a p95 route-decision time below 200 milliseconds under a defined load, 99.95% monthly availability for core orchestration, zero duplicate logical execution in fault-injection tests, and complete auditability of state transitions. These targets must be validated against realistic transaction volume, data freshness, partner behavior and budget. No measured result is claimed in this case study.

6.4 LIMITATIONS, RISKS AND MITIGATION

Risk / limitation	Potential impact	Mitigation
Stale treasury data	Route may be selected using unavailable liquidity	Freshness thresholds, reservations, source timestamps and route re-evaluation
FX quote expiry or market movement	Unexpected cost or failed conversion	Executable quotes, expiry checks, limits and controlled repricing
External rail unavailable	Delay or uncertain settlement	Circuit breaker, alternate eligible route, status inquiry and reconciliation
Duplicate submission	Double payment or financial loss	Idempotency keys, unique external references and deduplication
Distributed state inconsistency	Conflicting transaction status	Durable event log, state-machine validation, per-transaction ordering and reconciliation
Sanctions / AML false negative	Legal and reputational exposure	Approved screening vendors/rules, testing, monitoring and human escalation
False-positive screening	Unnecessary holds and delay	Tuned rules, case management, explainable matches and review SLAs
Cyberattack or credential compromise	Unauthorized transfers or data breach	Strong identity, least privilege, monitoring, key protection and incident response
Partner integration differences	Long onboarding and maintenance effort	Versioned adapters, conformance tests and partner certification
Regulatory or data-localization conflict	Launch blocked or data transfer restricted	Jurisdictional legal review, regional deployment and data-minimization controls
Over-optimization for cost	Route may increase risk or reduce reliability	Hard constraints, multi-objective scoring and governance-approved weights
Settlement finality limitations	A completed software step may not mean final funds transfer	Use explicit status semantics and rail-confirmed settlement evidence

6.5 Failure Recovery Strategy

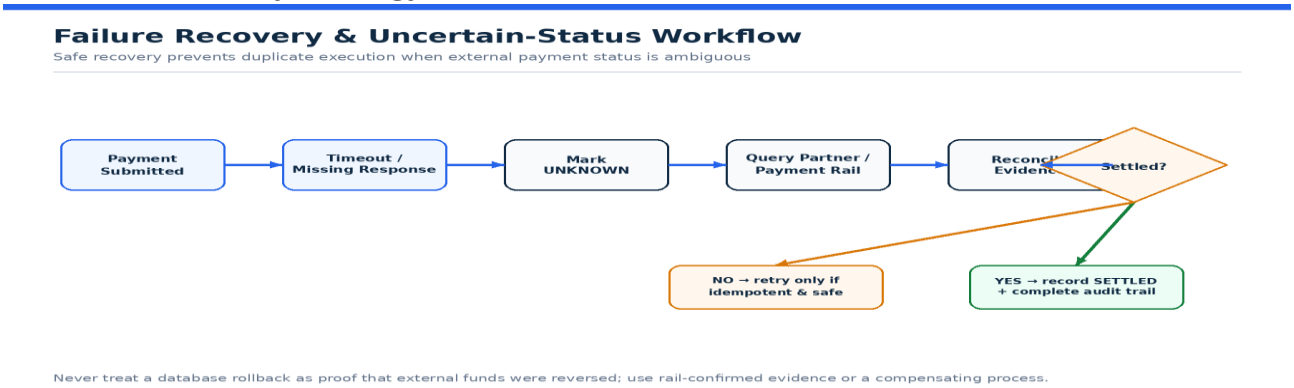


Figure 6.1. Failure Recovery and Uncertain-Status Workflow

A timeout should place the transaction into an uncertain or pending state, not automatically mark it failed. The system queries the partner or rail using the external reference and reconciles evidence. It retries only when the operation is known to be safe and idempotent. If funds have already settled, a database rollback cannot reverse the real-world payment; a return or compensating transaction may be required and must follow the relevant institution's rules.

7. FEASIBILITY ANALYSIS

7.1 Technical Feasibility

The solution is technically feasible as a phased integration platform using established software patterns: API gateways, event-driven services, replicated databases, workflow/state machines, graph algorithms, security controls, observability and payment-system adapters. The difficult work is not the route-scoring algorithm alone; it is reliable access to timely balances and quotes, partner interoperability, consistent transaction semantics, compliance integration and safe recovery after ambiguous external outcomes.

Technical area	Feasibility assessment	Conditions / dependencies
Graph routing	Feasible with standard shortest-path or constrained optimization techniques	Accurate edge costs, hard constraints and refreshed inputs
Distributed ledger/state	Feasible with durable database transactions and event log	Clear ownership of state, ordering, idempotency and reconciliation
High availability	Feasible with redundant deployments and failover	Multi-region design, tested recovery and partner availability
Compliance	Feasible through approved screening services and policy engine	Licensing, list updates, governance and human-review process
FX and liquidity	Feasible if providers expose usable APIs or feeds	Executable quotes, freshness, limits and reservation support
Integration	Moderate-to-high complexity	Bank-specific adapters, certification and contract testing

7.2 Operational Feasibility

Operational feasibility depends on banks and internal teams adopting shared workflows. Treasury must define balance freshness and route-risk limits; compliance must approve screening rules and escalation; operations must own exception queues and reconciliation; and engineering/SRE must operate monitoring, incident response and disaster recovery. A pilot should begin with a limited number of institutions, currencies and corridors. The team should conduct training, simulate incidents, agree on support coverage and document responsibilities before expanding.

7.3 Operational Readiness Checklist

- Named owner for every exception category and defined escalation window.
- Partner onboarding, credential rotation and connection-health monitoring documented.
- Runbooks for stale liquidity, expired FX quotes, sanctions holds, timeouts and duplicate-risk events.
- Daily reconciliation and exception aging reports available to operations.
- Incident communication plan and tested recovery objectives.
- Compliance sign-off and legal review for each pilot corridor.

7.4 Financial Feasibility

Financial feasibility should be evaluated through a corridor-specific business case. The main cost categories are platform engineering, security and compliance integration, hosting and observability, external data and FX services, bank/rail onboarding, certification, operational support, audit, and ongoing maintenance. Potential benefits include lower manual reconciliation cost, reduced avoidable intermediary fees where route alternatives exist, lower liquidity opportunity cost, fewer payment investigations, and improved customer retention. Savings must be measured against a baseline rather than assumed.

Cost / benefit category	Example calculation approach
Initial development	Engineering and architecture effort + testing + integration + security review
Recurring technology	Compute, database, messaging, monitoring, backup and disaster-recovery environments
External services	Screening lists, AML tools, FX feeds, payment connectivity and certification fees
Operations	Treasury, compliance, support, incident response and reconciliation staffing
Transaction-cost savings	Baseline fees and FX cost minus proposed-route fees and FX cost
Reconciliation savings	Baseline manual hours minus post-launch hours × fully loaded hourly cost
Liquidity benefit	Average reduction in idle prefunding × approved funding/opportunity-cost rate
Net annual benefit	Annual measured benefits − annual operating costs
Payback period	Initial investment ÷ average monthly net benefit, if net benefit is positive

No vendor quotations, transaction volumes, current fee schedule, staffing costs or measured baseline were supplied in the problem statement. Therefore, a defensible monetary ROI cannot be calculated yet. The recommended next step is a pilot with representative payment samples and a finance-approved measurement plan. The pilot should compare total cost, settlement-time distribution, reconciliation hours, liquidity utilization and failure rates.

7.5 Conclusion

The proposed Cross-Border Settlement & Liquidity Routing Protocol addresses the case study's central problem by combining dynamic route selection with liquidity and FX awareness, compliance gates, explicit transaction-state management, replicated audit records and safe recovery. The design follows software engineering principles through modularity, scalability, maintainability, security and reliability, while Agile Scrum with DevSecOps provides iterative development and controlled release. The intended outcomes are faster orchestration, better cost transparency, more efficient liquidity use and reduced manual reconciliation. These outcomes depend on partner connectivity, legal and regulatory approval, data quality, and measured pilot performance; sub-second final settlement cannot be guaranteed across every corridor.

REFERENCES AND DESIGN BASIS

[1] CSA10 / CSA1021 Software Engineering, "CO1 Assessment Tool 1 – Case Study Analysis," supplied assessment document, Annexure A.

[2] Financial Action Task Force (FATF), "International Standards on Combating Money Laundering and the Financing of Terrorism & Proliferation," FATF Recommendations, updated periodically.

[3] SWIFT, "Cross-border payments and transaction management," official product and documentation materials.

[4] Bank for International Settlements, Committee on Payments and Market Infrastructures, materials on cross-border payments, settlement and correspondent banking.

[5] ISO 20022, "Financial services — Universal financial industry message scheme," official standard overview.

[6] NIST, "Cybersecurity Framework 2.0," National Institute of Standards and Technology, 2024.

[7] OWASP Foundation, "Application Security Verification Standard (ASVS)," official project documentation.

[8] Scrum Guide, K. Schwaber and J. Sutherland, "The Scrum Guide," 2020.

Note: References 2–8 are suggested background sources for final verification and citation in a submitted academic version. Specific local regulations, technical standards and vendor capabilities should be confirmed for the jurisdictions and institutions selected for implementation.

3.6.1 Comparative Analysis of Delivery Methodologies

Method	Strength	Limitation for this case	Assessment
Waterfall	Clear sequential planning and fixed deliverables.	Late feedback may expose integration and compliance issues too late.	Suitable only where requirements are highly fixed.
Spiral	Strong risk analysis and prototyping.	Heavier management overhead for every incremental feature.	Useful for high-risk prototypes.
Pure DevOps	Strong automation, deployment and operational feedback.	Does not by itself provide product-planning and stakeholder-review structure.	Best used as an engineering complement.
Agile Scrum + DevSecOps	Incremental delivery, stakeholder review, automated quality/security and controlled release.	Requires disciplined governance; Agile does not remove approval requirements.	Selected approach.

Comparative conclusion: Agile Scrum with DevSecOps is the selected approach because the protocol combines changing compliance rules, partner integrations, security requirements and operational risks. It supports incremental delivery and stakeholder review while retaining controlled releases and governance. The proposed protocol improves coordination and control; it does not guarantee faster external settlement in every jurisdiction.